LLM Temelleri

İçindekiler

Tokenizer **3**

Context Length **3**

Model Türleri: Base (pt) ve Instruction-Tuned (it) 3

DataLoader **4**

Embedding **4**

Embedding Değerleri Nasıl Öğrenilir? 5

Hibrit ve Tam LLM Giriş Akışı 6

Self Attention **8**

Nasıl Çalışır? (Teknik Mantık) 8

Neden Önemli? 9

Örnek: "o" Kelimesi Kimi İşaret Ediyor? 9

Bağlam Penceresi Sınırı ve RAG 10

Nadir Kelimeler (Low-Frequency Token) Problemi 10

Genel Anlam (Eğitim) ve Dinamik Anlam (Çıkarım) 11

Sabit Embedding'den Dinamik Vektöre 12

Örnek: Next Token Prediction'da Self-Attention 13

Q, K, V Ağırlık Matrisleri (Kod) 15

Context Vector ve Sonraki Token Tahmini 16

Q ve K Çarpımı ve Ölçeklendirme 17

Şuan'a kadar ki yaptıklarımızın bir özeti: 19

Causal (Masked) Self Attention: **19**

Neden İhtiyaç Duyulur? 20

Nasıl Çalışır? (Teknik Süreç) 20

Bir Örnekle Açıklayalım 20

Dropout **21**

1. Attention Weights (Dikkat Ağırlıkları) Üzerinde 22

2. Context Vector (Bağlamsal Vektör) Çıkışında 22

Multi-Head Attention **22**

1. Neden "Multi" (Çoklu)? 23

Gerçek Modellerde Multi-Head Attention 23

MultiHead Attention Örneği: 26

Boyut Kıyaslaması: 29

Residual Connection ve Layer Normalization 30
1. Adım: Residual Connection (Kestirme Yol / Ekleme) 31
2. Adım: Layer Normalization (Katman Normalizasyonu) 31
Transformer Mimarisi (Encoder - Decoder) 31
1. Giriş Katmanı (Verinin Hazırlanması) 32
2. Sol Taraf: Encoder (Kodlayıcı) Katmanı 32
Decoder Katmanı 33
GPT Gibi Modeller: Sadece Decoder 34

Tokenizer

Tokenize etmek, Bir metnin, yapay zeka modelleri tarafından işlenebilecek daha küçük ve anlamlı parçalara bölünmesi işlemidir. Bu küçük parçaların her birine token denir. Tokenleştirme, kelimeleri veya karakterleri sayılara dönüştürmeden önceki köprü görevini görür. Tokenleştirme 3 türü var Kelime Düzeyinde, Harf Düzeyinde, Alt Kelime Düzeyinde.

Kelimelerin sonsuz kombinasyonları olabilir bunun için bir kelime sözlüğü vocab oluşturmak gerekir. Bunu yapan algoritmalar var. Ana mantık kelimelere sayı verilmesi. Subword tokenize de edilebilir mesela kitaplık kelimesini direkt bir token olarak almak yerine kitap ve lık 'ı ayrı token alırsak tasarruf etmiş oluruz. Büyük küçük harf, boşluk duyarlılığı vardır tokenizerlarda.

Detokenize ise yapay zeka modelinin ürettiği sayısal verilerin (tokenların) tekrar insanlar tarafından okunabilir, anlamlı bir metin haline getirilmesi sürecidir. Yapay zeka bir cevap üretirken kelimelerle değil, bu kelimelerin karşılığı olan sayısal değerlerle (token ID) çalışır. İşlem bittiğinde, karşınıza bir sayı yığını yerine anlamlı bir cümle çıkmasını sağlayan mekanizma detokenize işlemidir. Bu süreç genellikle bir **"Lookup Table"** (Arama Tablosu/Sözlük) kullanılarak gerçekleştirilir:

Context Length

- **Context Length** (Bağlam Uzunluğu), bir yapay zeka modelinin tek seferde işleyebileceği, "hafızasında" tutabileceği toplam veri miktarını ifade eder. Bunu, modelin aynı anda görebildiği bir "bakış açısı penceresi" veya kısa süreli hafızası gibi düşünebilirsin. Bir modelle konuşurken ona gönderdiğin mesajlar, modelin verdiği cevaplar ve varsa yüklediğin dosyaların/görsellerin tamamı bu pencerenin içine sığmak zorundadır. Context length sadece senin yazdığın kelimelerden oluşmaz; şu üç ana unsuru kapsar: Sistem Talimatları, Kullanıcı Girdisi, Modelin Cevabı. Dil modelleri, veriyi birer birim kaydırarak bir sonraki kelimeyi tahmin etmek üzere eğitilir; ancak bu tahmin yetenekleri, sahip oldukları **context length** (bağlam uzunluğu) kapasitesi ile sınırlıdır. Context length modelin işleyebileceği en uzun token sekansıdır.

Model Türleri: Base (pt) ve Instruction-Tuned (it)

- Google/gemma-3-27b-pt = : eğitilmiş bırakılmış sadece next token prediction var.
- .pt modelleri bir sonraki kelimeyi tahmin etmek üzere eğitilir. Onlara bir cümle verirseniz, o cümleyi en mantıklı şekilde tamamlamaya çalışırlar. Doğrudan bir asistan gibi davranmakta zorlanırlar. Genelde belirli bir alana yönelik (örneğin tıp veya hukuk) özelleştirilmek üzere bir "ham madde" olarak kullanılırlar. Eğer bir PT modeline "Türkiye'nin başkenti neresidir?" diye sorarsanız, size cevap vermek yerine "Türkiye'nin en kalabalık şehirleri nelerdir?" gibi benzer soruları sıralayabilir.

Çünkü o, elindeki veride bu tür soru listelerini görmüştür ve metni devam ettirmektedir.

- Google/gemma-3-27b-it: finetune edilmiş artık bize cevap verebilir.
- .it modelleri (Instruction Tuned / Talimat Eğitimi Almış) Model isminin sonundaki **IT**, modelin "Talimat Eğitimi" sürecinden geçtiğini ifade eder. Bu modeller, PT modellerinin üzerine inşa edilir. SFT (Supervised Fine-Tuning) ve genellikle RLHF (İnsan Geri Bildirimiyle Pekiştirmeli Öğrenme) süreçlerinden geçerler. Yani modele, "Bir soru sorulduğunda şu şekilde cevap ver" veya "Şu tür zararlı içerikleri üretme" gibi kurallar öğretilmiştir. Bir IT modeline bir görev verdiğinizde (örneğin "Bana bir Python kodu yaz"), doğrudan istenilen sonucu üretmeye odaklanır.

DataLoader

Deep Learning (Derin Öğrenme) projelerinde **DataLoader**, veri setinizi (dataset) modelin eğitime hazır olacağı şekilde küçük parçalara (batch) bölen, veriyi karıştıran (shuffle) ve işlemciyi (CPU) verimli kullanarak ekran kartına (GPU) sürekli veri akışı sağlayan bir mekanizmadır.

Kodlama aşamasında genellikle PyTorch veya TensorFlow/Keras gibi kütüphaneler aracılığıyla kullanılır.

Bir DataLoader temel olarak şu dört ana işlemi yönetir:

Batching (Gruplama), Shuffling (Karıştırma), Parallel Loading (Paralel Yükleme), Data Augmentation & Transformation.

DataLoader, verileri batch adı verilen gruplara bölerken, her bir verinin uzunluğunu modelin **Context Length** kapasitesine göre ayarlar. Eğer modelin context length sınırı 512 token ise, DataLoader her bir veri örneğini (metin veya görüntü yaması) bu sınıra göre kırpar (truncation) veya eksik kalanları özel işaretlerle (<pad>) tamamlar.

Embedding

Embedding, bilgisayarların kelimeleri, görselleri veya diğer verileri anlamlandırabilmesi için onları **sayı dizilerine (vektörlere)** dönüştürme işlemidir. Bilgisayarlar metinleri bizim gibi harf harf okuyarak değil, matematiksel değerler üzerinden işlem yaparak anlar. Ancak "elma" kelimesine sadece "1" demek yeterli değildir; çünkü bu sayı kelimenin anlamını veya diğer kelimelerle ilişkisini temsil etmez. Embedding teknolojisi burada devreye girer. Embedding'ler genellikle büyük dil modelleri eğitilirken **öğrenilir**. Model, milyonlarca metni okurken hangi kelimelerin hangi kelimelerle yan yana geldiğini analiz eder. Bu süreç sonunda, her kelimeye örneğin 768 veya 1536 adet sayıdan oluşan bir liste atanır. Bu sayılar kelimenin "cinsiyet", "canlılık", "resmiyet" gibi soyut özelliklerini temsil eder. Özetle embedding, ham veriyi yapay zekanın üzerinde matematik yapabileceği **"anlamsal bir**

haritaya" yerleştirme işlemidir. Örneğin Diyelim ki modelimiz dünyadaki tüm kelimeleri sadece 3 özelliğe göre puanlıyor olsun: **Canlılık**, **Yiyecek Olma** ve **Büyüklük**. Bu durumda kelimeler şu şekilde matematikselleşir:

Kelime	Canlılık Puanı	Yiyecek Puanı	Büyüklük Puanı
Kedi	0.95	0.01	0.20
Köpek	0.98	0.01	0.35
Elma	0.10	0.90	0.10
Karpuz	0.15	0.95	0.60

Kedi= [0.95, 0.01, 0.20]

Köpek = [0.98, 0.01, 0.35]

Elma=[0.10, 0.90, 0.10]

Karpuz = [0.15, 0.95, 0.60]

Buradaki matematikse anlamlaştırma şu şekilde olur:

1. **Mesafe (Benzerlik):** Bilgisayar "Kedi" ve "Köpek" vektörlerine baktığında, sayıların birbirine çok yakın olduğunu görür. Aralarındaki mesafe az olduğu için "Bu iki kelime benzer bir şeyi temsil ediyor olmalı" der.
2. **Yön (İlişki):** "Elma" ve "Karpuz" arasındaki farka baktığında ise "Yiyecek" puanlarının benzer ama "Büyüklük" puanlarının farklı olduğunu anlar.
3. **Çıkarım:** Sen modele "Kediye benzeyen bir şey getir" dediğinde, o aslında karmaşık bir **Pisagor teoremi** işleterek koordinat sisteminde "Kedi" noktasına en yakın olan "Köpek" noktasını bulup sana getirir.

Gerçek modellerde (örneğin senin çalıştığın projelerdeki LLM'lerde) bu puanlama 3 boyutta değil, genellikle **768 veya daha fazla boyutta** yapılır. Kelimeler; cinsiyet, zaman, duygu, teknik terim gibi binlerce farklı görünmez eksende puanlanarak kusursuz bir matematiksel anlam kazanır.

Embedding Değerleri Nasıl Öğrenilir?

Dil ne kadar karmaşıksa ve veri setiniz ne kadar büyükse, boyut sayısını o kadar yüksek tutabilirsiniz. Ancak boyut arttıkça işlem yükü de artar. Ancak ilginç olan husus Eksenlerin neyi temsil edeceğine biz karar vermiyoruz. İnsanlar olarak biz "Cinsiyet", "Zaman", "Sıcaklık" gibi somut eksenler bekleriz. Ancak model eğitimi bittiğinde, 158. eksenin neyi temsil ettiğini tam olarak bilemeyiz. Model, bu eksenleri verideki gizli desenleri (pattern)

çözmek için kendi kendine oluşturur. Bir eksen belki "resmiyet" seviyesini, diğeri "geçmiş zaman" ekini temsil ediyor olabilir ama bunlar matematiksel olarak soyuttur. Peki Kelimeler Eksendeki Değerlerini Nasıl Biliyor? Kelimeler başlangıçta değerlerini bilmezler.

1. Rastgele Başlangıç yapılır: Başlangıçta her kelimeye tamamen rastgele sayılardan oluşan bir vektör atanır. (Örn: "Elma" = [0.12, -0.5, 0.8...])
2. Tahmin: Modeli "boşluk doldurma" (Masked Language Modeling) veya "sonraki kelimeyi tahmin etme" göreviyle eğitmeye başlarız. * Örnek: "Maymun ağaçtan .... kopardı."
3. Hata Payı (Loss): Eğer model rastgele değerlerle "taş" tahminini yaparsa, biz ona "Hayır, cevap muz olmalıydı" deriz.
4. Geri Yayılım (Backpropagation): İşte kilit nokta! Model, hatasını düzeltmek için o kelimelerin embedding vektöründeki sayıları minik minik değiştirir. Birbirine benzer bağlamlarda (context) geçen kelimeler (örn: "muz" ve "elma"), zamanla bu devasa uzayda birbirine yakın koordinatlara sürüklenirler. Çünkü ikisi de "yemek", "koparmak", "lezzetli" gibi kelimelerle yan yana gelir.

Eğer daha önce hiç çalışılmamış bir dilde model eğitecekseniz şu yolu izlersiniz: Veri Toplama: O dilde yazılmış milyonlarca satır metin (kitap, haber, altyazı). Vocab Oluşturma: Byte Pair Encoding (BPE) gibi yöntemlerle kelime/ek kütüphanesini çıkarıp ID'leri verirsiniz. Embedding Katmanı: Rastgele sayılarla dolu bir tablo (Matris) oluşturursunuz. Self-Supervised Learning: Modeli "kendi kendine öğrenme" moduna bırakırsınız. Model, milyonlarca cümleyi okuya okuya hangi kelimenin hangi koordinatta durması gerektiğini (matematiksel benzerliklerden yola çıkarak) kendisi bulur. Özetle: Biz sadece "oyun alanını" (boyut sayısını) kuruyoruz ve "kuralları" (hata payı hesaplama) öğretiyoruz. Kelimelerin o koordinatlara yerleşmesi, tamamen istatistiksel bir "mahalleleşme" sürecidir.

Embedding boyutlarının **"Canlılık"** veya **"Büyüklük"** gibi isimleri yoktur. Bunlara **"Latent Dimensions" (Gizli Boyutlar)** denir. ID numaraları (452, 1208 vb.) sadece birer etikettir ve kelimeler arasındaki anlamsal ilişkiyi temsil etmez. Embedding katmanı, bu ID'leri yüksek boyutlu yoğun vektörlere dönüştürür.

Bu vektörlerin içindeki sayılar rastgele başlar ve model eğitildikçe güncellenir. Benzer anlamdaki kelimeler (örneğin "kral" ve "kraliçe"), bu çok boyutlu uzayda birbirine yakın konumlara yerleşir. Sıralama şöyledir:

Hibrit ve Tam LLM Giriş Akışı

1. Token ID'si Alınır

Ham metin (örn: "Kedi") parçalara ayrılır ve sözlükteki ID'si bulunur.

- "Kedi" ID: 540

2. Word Embedding (Anlamsal Vektör)

ID'ye karşılık gelen vektör tablodan çekilir.

- **Başlangıç:** Tamamen rastgele (Random) değerler.
- **Görev:** Kelimenin sadece "ne" olduğu bilgisini taşır.

3. Positional Bilgisinin Hazırlanması (İki Seçenekli/Hibrit)

Burada model mimarisine göre iki yoldan biri seçilir:

- **A) Learned (Random) Table:** Bilgisayarın belleğinde [Max_Uzunluk x Vektör_Boyutu] boyutunda rastgele sayılardan oluşan bir tablo daha vardır. Kelime 3. sıradaysa, bu tablonun 3. satırı çekilir.
- **B) Sinüsoidal (Fixed):** Bir tablo yerine, o anda kelimenin sırasına (3. sıra) göre Sinüs ve Kosinüs formülleriyle bir vektör hesaplanır.

4. Vektörel Birleştirme (Toplama)

Kelime vektörü ile seçilen pozisyon vektörü toplanır.

Kritik Detay: Artık elimizdeki vektör hem "Kedi" anlamını taşıyor hem de üzerine "3. sıra" imzası atılmış durumda.

5. Forward Pass (Modelin İçine Giriş)

Bu toplam vektör, modelin ana motoru olan Transformer bloklarına (Self-Attention katmanına) girer.

6. Tahmin ve Kayıp (Loss) Hesaplama

Model bir tahminde bulunur ve doğru cevaptan ne kadar uzak olduğu (Loss) hesaplanır.

7. Geri Yayılım (Backpropagation - Öğrenme Aşaması)

Hata payı geriye doğru gönderilirken güncelleme işlemi şöyle ayrışır:

- **Word Embedding Katmanı:** GÜNCELLENİR (Kelime vektörleri daha iyi anlam kazanır).
- **Modelin İç Ağırlıkları:** GÜNCELLENİR.
- **Pozisyon Kısmı (İşte fark burada):**
    - Eğer **Random (Learned)** kullandıysan: Pozisyon tablosu da **GÜNCELLENİR.** Model "3. sırada olmanın" ne demek olduğunu sayılarla öğrenir.
    - Eğer **Sinüsoidal** kullandıysan: Pozisyon değerleri **GÜNCELLENMEZ.** Çünkü onlar sabit bir matematiksel formüldür. Model, bu sabit kurala uyum sağlamayı öğrenir.

- **Eğer RoPE (Rotary) kullandıysan:** Bu sefer kelimelerin hem cümle içindeki sırasını hem de birbirlerine olan mesafelerini kullanır. Tıpkı Sinüsoidal yöntemdeki gibi, döndürme miktarları (açılar) matematiksel bir kurala bağlıdır ve **GÜNCELLENMEZ.** Ancak, model bu döndürme işleminin sonucunda oluşan "göreceli farkları" yorumlamayı çok daha hızlı öğrenir. En çok kullanılan yöntem budur. Kelimeler arasındaki mesafeyi sadece mutlak yerleriyle değil, birbirlerine olan uzaklıklarıyla (açı farkıyla) kavrar; bu da modelin metin içindeki mantıksal bağları çok daha hassas kurmasını sağlar. Matematiksel yapısı gereği, Transformer'ın ana motoru olan "Attention" (Dikkat) hesaplamasına kelimelerin mesafe bilgisini doğrudan ve kayıpsız bir şekilde enjekte eder.

Yani embedding'ler, üzerlerine konum bilgisi binmiş haldeki performanslarına göre eğitilirler.

Self Attention

Self-attention (öz-dikkat), bir dizideki (örneğin bir cümledeki) her bir öğenin, aynı dizideki diğer tüm öğelerle olan ilişkisini hesaplayan ve her birinin önem derecesini belirleyen bir mekanizmadır. Özellikle **Transformer** mimarisinin (GPT, BERT gibi modellerin temeli) kalbidir.

Basitçe ifade etmek gerekirse; bir kelimenin, cümledeki diğer hangi kelimelerle "alakalı" olduğunu anlamasını sağlar.

Nasıl Çalışır? (Teknik Mantık)

Self-attention mekanizması, her bir girdi (token) için üç ana vektör oluşturur:

1. **Query (Sorgu - Q):** "Ben neyi arıyorum?" (Mevcut kelimenin diğerlerine sorduğu soru).
2. **Key (Anahtar - K):** "Ben ne sunuyorum?" (Diğer kelimelerin eşleşebileceği etiket).
3. **Value (Değer - V):** "Eğer benimle eşleşirsen, sana hangi bilgiyi vermeliyim?"

Süreç matematiksel olarak şu adımlarla ilerler:

- **Skaler Çarpım (Dot Product):** Her bir Query ile tüm Key'ler çarpılarak bir "benzerlik puanı" elde edilir.
- **Softmax:** Bu puanlar normalize edilir (0-1 arasına çekilir), böylece toplamları 1 olan ağırlıklar oluşur.

- **Ağırlıklı Toplam:** Bu ağırlıklar Value vektörleriyle çarpılır. Sonuçta, her kelime kendisi için en önemli olan diğer kelimelerin bilgisini daha yoğun içeren yeni bir temsil kazanır.

Neden Önemli?

- **Bağlam (Context) Yakalama:** "Banka" kelimesinin geçtiği bir cümlede, self-attention sayesinde model "para" veya "nehir kenarı" kelimelerine bakarak hangisinin kastedildiğini anlar.
- **Paralel İşleme:** RNN (Recurrent Neural Networks) gibi yapıların aksine, veriyi sırayla değil, bir bütün olarak işleyebilir. Bu da GPU'lar üzerinde çok daha hızlı eğitim yapılmasını sağlar.
- **Uzun Mesafe İlişkileri:** Cümlenin başındaki bir kelime ile sonundaki bir kelime arasındaki ilişkiyi, aradaki mesafe ne olursa olsun tek bir adımda kurabilir.

Örnek: "o" Kelimesi Kimi İşaret Ediyor?

Self-attention mekanizmasını daha iyi kavramak için şu cümleyi ele alalım:

"Bankadaki adam dosyaları imzaladı çünkü o çok meşguldü."

1. Odaklanma (Query-Key Eşleşmesi)

Model, "o" kelimesine geldiğinde (bu bizim **Query**'miz olur), cümledeki diğer tüm kelimelere (bunlar da **Key**'lerimizdir) şu soruyu sorar: *"Benimle en çok hanginiz alakalı?"*

- **"o" vs "Bankadaki":** Alaka düşük.
- **"o" vs "Dosyaları":** Gramer olarak mümkün olsa da "meşgul olan dosya" mantığı zayıf.
- **"o" vs "Adam":** Alaka çok yüksek. "Meşgul olmak" eylemi bir kişiyle (adam) doğrudan ilişkilidir.

2. Ağırlıklandırma (Attention Scores)

Model her kelimeye bir puan verir. "o" kelimesi için hesaplanan dikkat (attention) ağırlıkları yaklaşık olarak şöyle görünebilir:

- **Adam:** 0.85
- **Bankadaki:** 0.05
- **Dosyaları:** 0.08
- **İmzaladı:** 0.02

Bu puanlar kullanılarak, "o" kelimesinin o cümledeki nihai matematiksel temsili (vektörü) oluşturulur. "o" artık sadece bir zamir değildir; içeriğinin **%85'i "adam"** kelimesinden gelen bilgilerle güncellenmiştir.

Bağlam Penceresi Sınırı ve RAG

Her modelin (örneğin GPT-4, Llama-3 veya kendi eğittiğin bir Transformer) bir **maksimum token sınırı** vardır (örneğin 8k, 32k veya 128k token).

- Eğer senin 8000 sayfan 10 milyon token ediyorsa ve modelin kapasitesi 8 bin tokensa, model sadece o anki 8 bin token içindeki ilişkileri hesaplayabilir.
- Self-attention mekanizması, bu pencerenin dışındaki hiçbir şeyi "görmez" ve onlarla alaka kuramaz.
- Her bir batch içindeki örnekler bile birbirinden bağımsızdır. Yani Batch 1'deki bir cümle ile Batch 2'deki bir cümle arasında self-attention kurulmaz. Self-attention sadece her bir örneğin (sequence) kendi içindeki token'lara bakması için tasarlanmıştır.

Örneğin 8000 sayfanın tamamındaki bilginin birbirini etkilemesini istiyorsan, karşımıza çıkan çözümlerden biri şudur:

> **RAG (Retrieval-Augmented Generation):** 8000 sayfayı küçük parçalara bölüp bir vektör veritabanına atarsın. Kullanıcı bir soru sorduğunda, o soruyla en alakalı 3-5 sayfayı bulup "context" olarak modele verirsin. Model yine sadece o 3-5 sayfayı (self-attention ile) görür ama sanki hepsini biliyormuş gibi davranır.

Nadir Kelimeler (Low-Frequency Token) Problemi

Mesela bir kelime bir batch'e geçiyor ama anlamı self-attention katmanında diğer kelimelerle ilişkisi tam oturmadı

Bu durum, derin öğrenmede **"Low-Frequency Token"** (düşük frekanslı kelime) veya **"Out-of-Distribution"** problemi olarak adlandırılır. Eğer bir kelime (token) veri setinde çok nadir geçiyorsa, model onun diğer kelimelerle olan ilişkisini (yani Wq, Wk, Wv matrislerini o kelime için nasıl optimize edeceğini) tam öğrenemez.

Bu sorunu çözmek için kullanılan temel yöntemler şunlardır:

1. Subword Tokenization (Kelimeyi Parçalara Bölme)

Modern modeller (GPT, BERT, Llama) kelimeleri tam halleriyle değil, **BPE (Byte Pair Encoding)** veya **WordPiece** gibi yöntemlerle alt parçalara böler.

- **Örnek:** Eğer "elektromanyetikleşemeyebileceklerimizdenmişsinizcesine" gibi çok nadir bir kelime gelirse, model bunu "elektro", "manyetik", "leş", "ebil" gibi daha sık rastlanan parçalara böler.

- **Çözüm:** Model "elektro" ve "manyetik" parçalarını diğer batch'lerde binlerce kez gördüğü için, nadir kelimenin *köklerinden* anlamı kurtarır.

2. Pre-trained Embeddings (Hazır Kelime Vektörleri)

Eğer modeli sıfırdan eğitiyorsan ve verin azsa, kelime temsilcilerini (embeddings) rastgele başlatmak yerine **Word2Vec** veya **GloVe** gibi önceden devasa verilerle eğitilmiş hazır vektörlerle başlatabilirsin.

- **Çözüm:** Model bu kelimeyi senin batch'inde ilk kez görse bile, kelimenin genel anlamı vektör uzayında zaten "doğru yere" yakın başlar.

3. Data Augmentation (Veri Artırımı)

Madem bu kelime diğer batch'lerde yok, onu biz ekleyebiliriz.

- **Synonym Replacement:** O kelimenin geçtiği cümleleri kopyalayıp, benzer anlamlı kelimelerle veya aynı kelimeyi farklı bağlamlarda kullanarak yeni batch'ler oluşturulur.
- Senin **Stable Diffusion** ve **LoRA** tecrübendeki mantık burada da geçerli; az veri varsa sentetik olarak çeşitlendiriyoruz.

4. Dropout ve Regularization

Eğer bir kelime nadir geçiyorsa, model o kelimeyi sadece o spesifik cümledeki kelimelerle "ezberleme" (overfitting) eğilimine girer.

- **Dropout:** Eğitim sırasında attention ağırlıklarının bir kısmını rastgele sıfırlayarak modelin tek bir kelimeye aşırı güvenmesini engelleriz. Bu, modelin daha "genel" bir ilişki kurmaya zorlanmasını sağlar.

5. Transfer Learning (En Yaygın Çözüm)

Günümüzde kimse bir Transformer modelini (eğer çok spesifik bir tıp/hukuk dili değilse) sıfırdan eğitmiyor.

- **Çözüm:** Halihazırda milyarlarca sayfa okumuş bir modeli (base model) alıp kendi küçük veri setinle **Fine-Tuning** yaparsın. Bu sayede model, senin batch'inde az geçen kelimeyi zaten internetteki diğer milyarlarca cümleden tanıyor olur

Genel Anlam (Eğitim) ve Dinamik Anlam (Çıkarım)

Eğitim sırasında evet, o kelimenin genel anlamı (weight/embedding) tüm batch'lerden gelen bilgiyle güncellenir. Ancak **çalışma anında (inference)**, bir kelimenin anlamı sadece içinde bulunduğu cümledeki (sequence) diğer kelimelerle belirlenir.

Konuyu üç seviyede netleştirelim:

1. Eğitim Aşaması: "Genel Anlamın" Şekillenmesi

Sen modeli eğitirken, aynı "banka" kelimesi Batch 1'de "para" ile, Batch 10'da ise "nehir" ile yan yana gelebilir.

- Modelin **Embedding Katmanı** ve **Attention Ağırlıkları (Wq, Wk, Wv)**, her iki batch'ten gelen hata payına (loss) göre güncellenir.
- Bu süreçte model, "banka" kelimesinin iki farklı bağlamda da kullanılabileceğini öğrenir. Yani modelin **sabit parametreleri** (beyni), tüm batch'lerin toplam tecrübesiyle değişir.

2. Çıkarım (Inference) Aşaması: "Dinamik Anlamın" Oluşması

Model eğitildikten sonra dondurulur (weights artık değişmez). Şimdi modele bir cümle veriyorsun:

"Vaktim kısıtlı olduğu için bankadaki işlemlerimi hızlıca hallettim."

Bu noktada self-attention devreye girer. Kelimenin genel anlamı ("banka" hem nehir hem para olabilir bilgisi) modelde zaten vardır. Ancak self-attention mekanizması:

- Sadece **o anki dizideki** "işlem", "vakit", "halletmek" kelimelerine yüksek puan verir.
- Böylece "banka" kelimesinin o cümledeki temsili (vektörü), diğer batch'lerdeki nehir anlamından **tamamen bağımsız** olarak, o anki "finans" anlamıyla güncellenir.

Sabit Embedding'den Dinamik Vektöre

Modelinin içinde nn.Embedding adında dev bir tablo (lookup table) bulunur. Sen "BANKA" dediğinde, model bu tabloya gider ve o kelimeye karşılık gelen **sabit sayı dizisini** (örneğin 512 boyutlu bir vektör) çeker.

- Bu liste (vektör) **statiktir**.
- Senin .pt dosyanın içinde "BANKA" için ayrılmış tek bir satır vardır.
- Cümle ne olursa olsun, "BANKA" kelimesi modelin kapısından içeri her zaman **aynı** listeyle girer.

2. Aşama: Self-Attention Katmanı (Dinamikleşme)

Sihir tam burada gerçekleşir. O sabit liste, Self-Attention katmanına girdiğinde artık yalnız değildir; yanında cümlenin diğer kelimeleri (örneğin "nehir" veya "para") de vardır.

Self-Attention şu işlemi yapar:

1. "BANKA" vektörünü alır.
2. Cümledeki diğer kelimelerin vektörlerine bakar.

3. Eğer yanında "para" varsa, "BANKA" vektörünün içindeki belirli sayıları (finansla ilgili olanları) baskın hale getirir.
4. Sonuç olarak, katmandan çıkan vektör artık o **sabit liste değildir**.

Özetle: Çıkan şey, başlangıçtaki sabit listenin üzerine cümlenin geri kalanından gelen bilgilerin "eklenmiş/yoğurulmuş" halidir.

Örnek: Next Token Prediction'da Self-Attention

Diyelim ki modelin şu ana kadar ürettiği cümle şu:

"Mühendis, bozulan bilgisayarı tamir etmek için tornavidayı..."

Şimdi model bir sonraki kelimeyi (next token) tahmin edecek. Seçenekler: **"aldı"**, **"fırlattı"**, **"yedi"**.

1. Adım: Kelimelerin Hazırlanması (Embedding)

Model "tornavidayı" kelimesine geldiğinde, elindeki tüm kelimeleri (Mühendis, bozulan, bilgisayarı...) sabit listeler (embedding) olarak sisteme alır.

2. Adım: Self-Attention Sorgusu (Query-Key-Value)

Modelin "tornavidayı" kelimesi için bir sonraki adımı belirlemesi gerekir. Burada self-attention şu kritik soruları sorar:

- **Query (Sorgu):** "Elimde bir 'tornavida' var, bununla ne yapılır?"
- **Key (Anahtar):** Cümledeki diğer kelimeler kendilerini tanıtır:
    - "Mühendis": Ben bir insanım, alet kullanabilirim.
    - "Tamir etmek": Ben bir amacım, tornavidayla uyumluyum.
    - "Bilgisayarı": Ben tamir edilecek nesneyim.
- **Value (Değer):** Bu kelimelerin taşıdığı asıl anlamlar.

3. Adım: Dikkat Puanlarının Hesaplanması

Model, "tornavidayı" kelimesinden bir sonraki kelimeye geçerken geçmişteki kelimelere şu puanları verir:

Geçmiş Token	Dikkat Puanı (Attention Weight)	Neden?
Tamir etmek	0.60 (Yüksek)	Tornavida ve tamir birbiriyle doğrudan ilişkili.
Mühendis	0.25 (Orta)	Özneyi hatırlamak önemli.
Bilgisayarı	0.10 (Düşük)	Nesne zaten belirlenmiş.
Bozulan	0.05 (Çok Düşük)	Bu bilgi şu anki eylem için çok kritik değil.

4. Adım: Tahmin (Prediction)

Self-attention sayesinde "tornavida" vektörü, **"tamir etmek"** ve **"mühendis"** kavramlarıyla harmanlanır.

- Eğer self-attention olmasaydı, model sadece "tornavidayı..." kelimesine bakıp rastgele bir fiil (mesela "yedi") seçebilirdi.
- Ancak self-attention sayesinde model, tornavidanın bir **mühendis** tarafından **tamir** için kullanıldığını "hatırlar".

Bu harmanlanmış bilgi, modelin en üst katmanındaki (Softmax) olasılık dağılımını değiştirir:

- **aldı:** %85
- **çıkardı:** %10
- **unuttu:** %4
- **yedi:** %1

Özetle Next Token Prediction'da Self-Attention:

Sıradaki kelimeyi tahmin etmeden hemen önce, **"Arkama bakıp hangi kelimeleri dikkate almalıyım?"** sorusunu çözen mekanizmadır. Model, geçmişteki kelimelerin sabit listelerini alır, onları birbirine çarparak o ana özel "bağlamsal bir özet" çıkarır ve bu özete dayanarak sıradaki kelimeyi (token) seçer.

Q, K, V Ağırlık Matrisleri (Kod)

```
q_weights = torch.nn.Linear(4, 3, bias=False)
k_weights = torch.nn.Linear(4, 3, bias=False)
v_weights = torch.nn.Linear(4, 3, bias=False)

q_of_sentence = q_weights(sentence_meanings_with_atention_context)
k_of_sentence = k_weights(sentence_meanings_with_atention_context)
v_of_sentence = v_weights(sentence_meanings_with_atention_context)
print(q_weights.weight)

q_of_sentence.shape, k_of_sentence.shape, v_of_sentence.shape
```

Kodun içindeki eşleşmeler şu şekildedir:

- **q_weights (Wq):** Query (Sorgu) ağırlık matrisidir. Girdi matrisini (E+P) sorgu uzayına projekte eder.
- **k_weights (Wk):** Key (Anahtar) ağırlık matrisidir. Girdiyi anahtar uzayına taşır.
- **v_weights (Wv):** Value (Değer) ağırlık matrisidir. Girdiyi, dikkat puanlarıyla çarpılacak olan "anlam" uzayına taşır.

Wq, Wk, Wv Matrisleri: Burası **"muhakeme" ve "uzmanlık" bilgisinin** saklandığı yerdir. Modelin eğitim sırasında öğrendiği her şey bu matrislerin içindeki sayılarda (weight) saklanır.

- Bu matrisler, "Hangi kelime hangi durumda daha önemlidir?" veya "Kelimeler arasındaki gizli ilişkiler nasıl kurulur?" sorularının yanıtlarını bilirler.

Kodun Teknik Analizi

1. **torch.nn.Linear(4, 3, bias=False)**:
    - **4:** Senin girdi embedding boyutun (D_model). Yani her kelime 4 sayıdan oluşan bir vektör. Senin her bir kelimen (token), 4 boyutlu bir vektör olarak tanımlanmış.
    - **3:** Çıkış boyutun (d_k). Modelin, bu 4 sayılık bilgiyi 3 sayılık daha "öz" bir temsil içine sıkıştıracağını veya dönüştüreceğini gösteriyor. Bu sayede model, 4 boyutlu ham bilginin içinden en önemli 3 özelliği seçip "Ben şu an bu 3 özelliği arıyorum" diyebilir.
    - **bias=False**: Bu çok kritik; çünkü Attention mekanizması saf matris çarpımı üzerine kuruludur. Ekstradan bir kaydırma (bias) değeri eklenmez.
2. **q_of_sentence (Q)**:

- q_weights(sentence_meanings_with_attention_context) satırı, ham cümle temsilini alıp onu Wq ile çarparak **Query** vektörlerini oluşturur. Artık elinde "ben neyi arıyorum?" sorusunu soran vektörler var.

3. **print(q_weights.weight):**
   Burada ekrana yazdırdığın şey, modelin eğitim sırasında güncelleyeceği o eğitilebilir parametrelerin (matris elemanlarının) ta kendisidir.

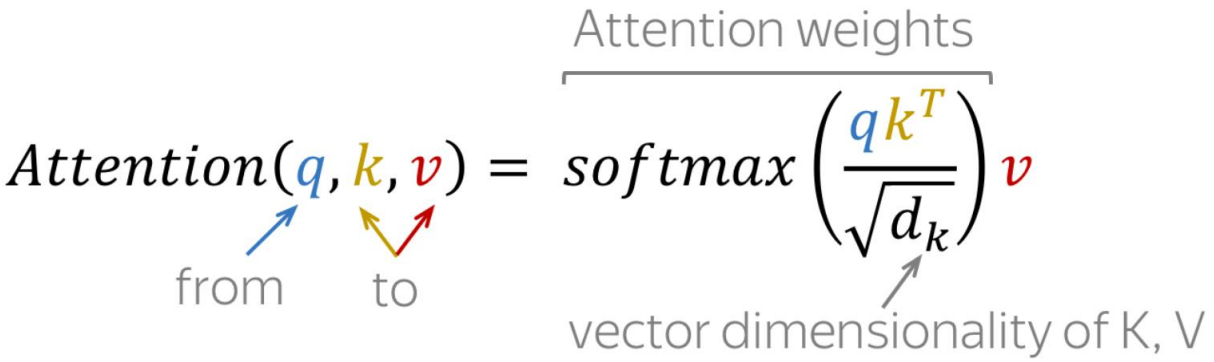
Sonraki Adımlar ise şu şekilde:

q_of_sentence ile k_of_sentence'ın transpozunu çarparak (dot product) o ilk başta konuştuğumuz benzerlik matrisini elde etmek.

Çıkan sonuçları Softmax'ten geçirmek.

Elde edilen ağırlıkları v_of_sentence ile çarparak **Weighted Sum** (Ağırlıklı Toplam) işlemini bitirmek.

Context Vector ve Sonraki Token Tahmini




```
#Cümledeki her kelimenin, diğer tüm kelimelerle olan ham "benzerlik" veya "alaka" puanını hesaplar.
attention_scores = q_of_sentence @ k_of_sentence.T #Benzerlik Matrisi (attention_scores)

#Bölme işlemi, dot product sonuçlarının çok büyümesini ve gradyanların patlamasını (Softmax'in doyuma ulaşmasını) engeller.
#Softmax: Bu ham puanları $0$ ile $1$ arasına çeker ve her satırın toplamını 1 yapar.
attention_weights = torch.softmax(attention_scores / k_of_sentence.shape[-1] ** 0.5, dim=1)#Ölçekleme ve Olasılığa Dönüştürme

#Hesapladığın o hassas ağırlıkları, gerçek bilgi/anlamı taşıyan Value (V) matrisi ile çarpıyorsun.
#Bu adımda her kelime, cümledeki diğer kelimelerden ne kadar bilgi alacağına karar verir.
#context_vector, her kelimenin orijinal embedding'inden çok daha akıllı,
#çevresindeki kelimelerin anlamını da içine çekmiş "bağlamsal" (context-aware) halidir.
context_vector = attention_weights @ v_of_sentence #Bilginin Harmanlanması
context_vector
```

Kelimeler önce birbirlerine soru sordu (**Q**), etiketlerini sundu (**K**) ve aralarındaki alaka belirlendi (**Weights**). Son olarak bu alakaya göre her kelime birbirinden bir miktar anlam (**V**) alarak kendi yeni anlamını (**Context Vector**) oluşturdu.

Bu işlemden sonra, model artık "banka" kelimesinin o cümledeki gerçekte neyi temsil ettiğini (finans mı, doğa mı) bu context_vector içindeki sayılar sayesinde çok iyi biliyor olur.

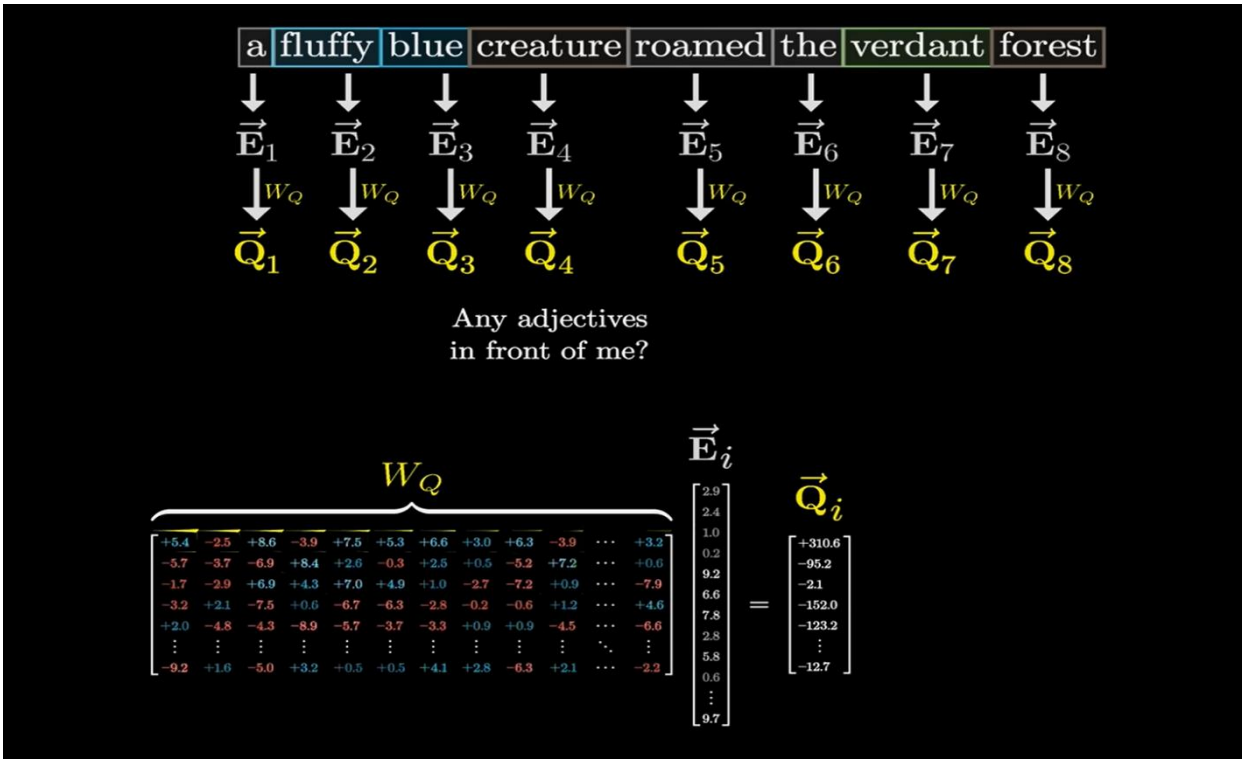
Eğitimden sonra sabitlenmiş olan Wq, Wk ve Wv matrisleri, o anki (en son gelen) kelimeye ve kendinden önceki tüm kelimelere uygulanır. Her bir kelime bu "filtrelerden" geçerek kendine has bir **Sorgu (Q)**, **Anahtar (K)** ve **Değer (V)** kimliği kazanır. "O anki kelime" (en son kelime), kendi **Sorgu (Q)** vektörüyle geçmişteki tüm kelimelerin **Anahtar (K)** vektörlerine şu soruyu sorar:

"Bir sonraki kelimeyi doğru tahmin edebilmem için hanginizde daha kritik bir bilgi var?"

Bu filtreleme sonucunda, o anki kelimeyle en alakalı olan geçmişteki kelimeler (yüksek attention weight alanlar) öne çıkar. Örneğin, "içti" kelimesi için "su" kelimesi çok yüksek bir filtre puanı alır. Bu puanlar kullanılarak kelimelerin **Değer (V)** vektörleri harmanlanır. Artık elimizde sadece "içti" kelimesi değil, "neyin içildiği" bilgisini de içinde taşıyan çok güçlü bir **bağlamsal vektör** vardır. İşte bu **bağlamsal vektör**, modelin en sonundaki "tahmin kafasına" (Output Head) gider. Model, bu zenginleştirilmiş bilgiye bakarak;

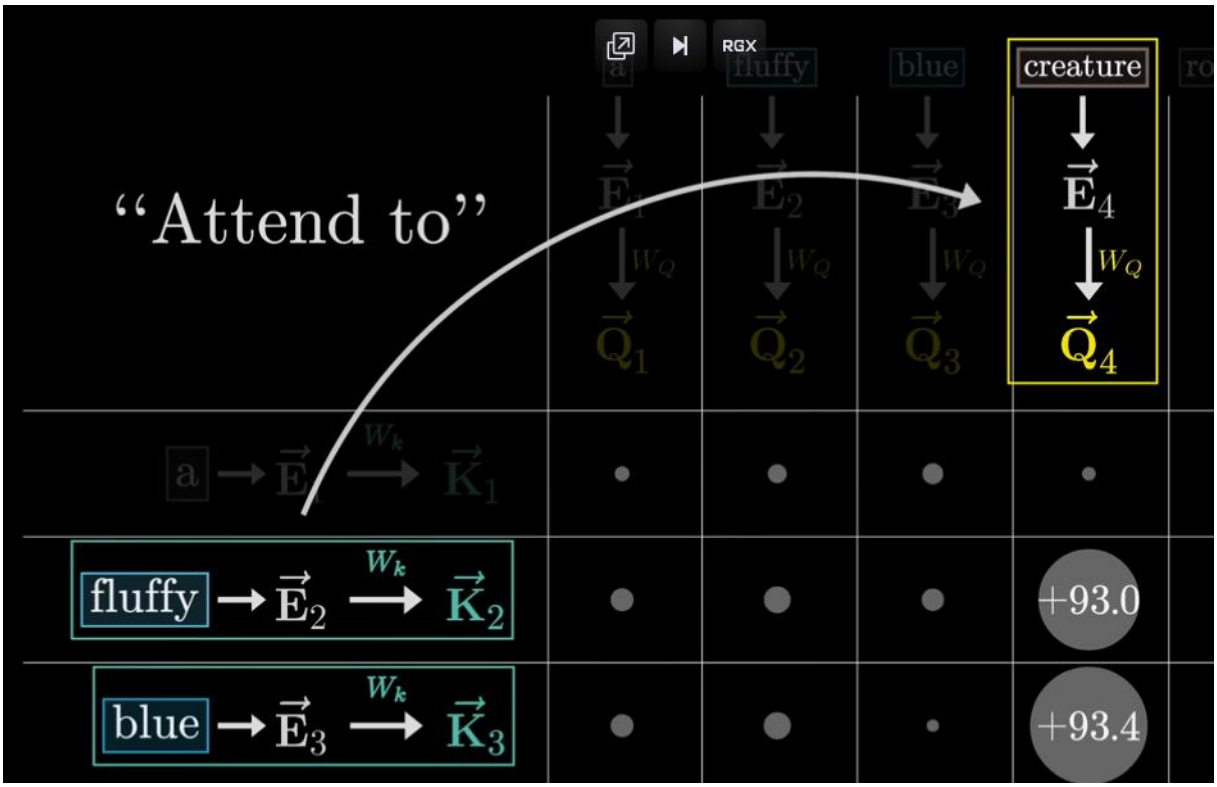
- *"Madem su içildi, o zaman bir sonraki token 'serinledi' veya 'bardak' olabilir"* diyerek olasılıkları hesaplar.

Q ve K Çarpımı ve Ölçeklendirme

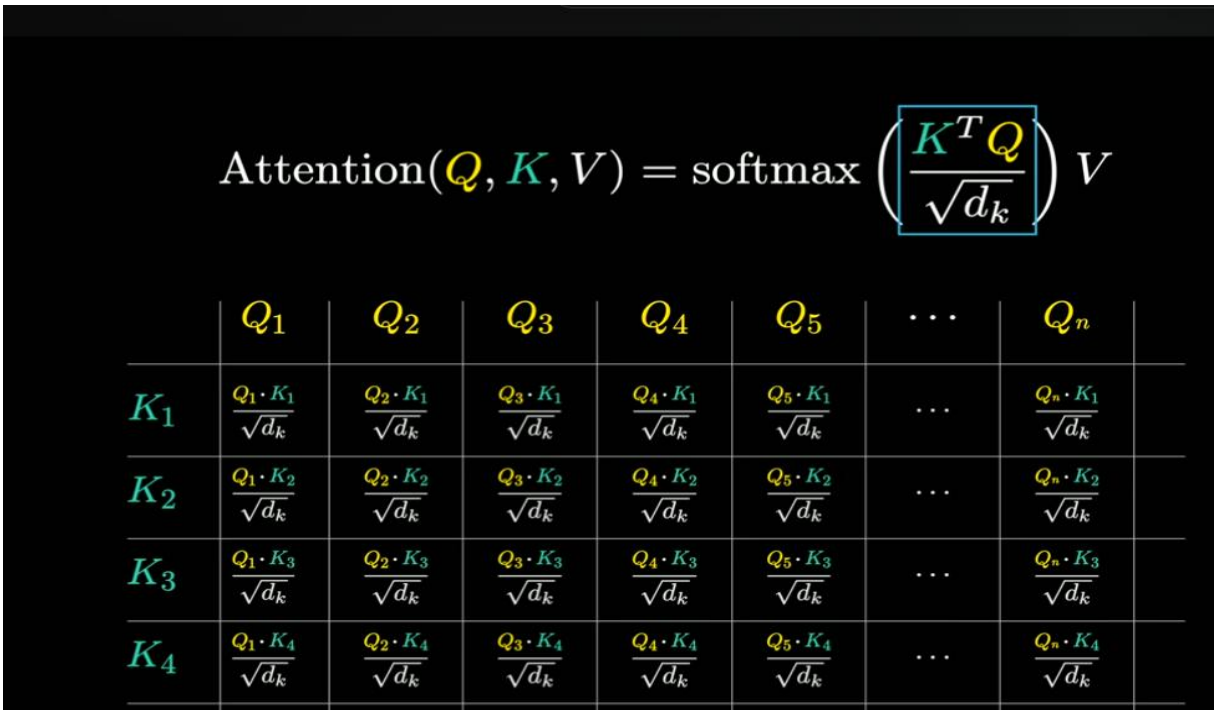




Burada Wq ve Wk tuneable değiştireleilbir parametlerelerdir. Çarpımlarının hangisi büyükse onu alcaz yukarıdaki fotoda noktası büyük olan çarpımın büyük olduğu anlamına geliyor.



Yukarıdaki örnek gibi. Sonrasında Q ve K transpozu çarpılır.



Nokta çarpımı dk ile bölerek varyansı tekrar **1**'e çekmiş (yani ölçeklendirmiş) oluyoruz.

Şuan’a kadar ki yaptıklarımızın bir özeti:

20.vdeionun sonuna tekrar bak.

```python
class UstaModel(nn.Module):
  def __init__(self, vocab_size, embedding_dim, context_length):
    super().__init__()
    self.embedding = nn.Embedding(vocab_size, embedding_dim)
    # position embedding but not being used in the forward pass
    # it is just for educational purposes
    self.pos_embedding = nn.Embedding(context_length, embedding_dim)
    self.get_pos = get_rotary_position_encoding
    self.self_attention = UstaSelfAttention(embedding_dim, embedding_dim)

  def forward(self, x):
    x = self.embedding(x) # dictionary meaning of the tokens (words)
    x = self.get_pos(x) # meaning of the tokens in the sentence according to their position
    x = self.self_attention(x)
    return x
```

Causal (Masked) Self Attention:

Causal self-attention (veya sıkça kullanılan adıyla **Masked Self-Attention**), özellikle GPT gibi üretken (generative) modellerin temelini oluşturan mekanizmadır.

Normal self-attention'da her kelime cümledeki tüm kelimelere (geçmişe ve geleceğe) bakabilirken; **causal self-attention**, modelin bir kelimeyi işlerken kendisinden sonra gelen kelimeleri "görmesini" engeller.

Neden İhtiyaç Duyulur?

Sıradaki kelime tahmini (**Next Token Prediction**) görevinde modelin kopya çekmemesi gerekir. Eğer model 3. kelimeyi tahmin etmeye çalışırken 4. ve 5. kelimelerin bilgisine sahip olursa, öğrenme gerçekleşmez. Bu yüzden gelecekteki bilgi bir "maske" ile kapatılır.

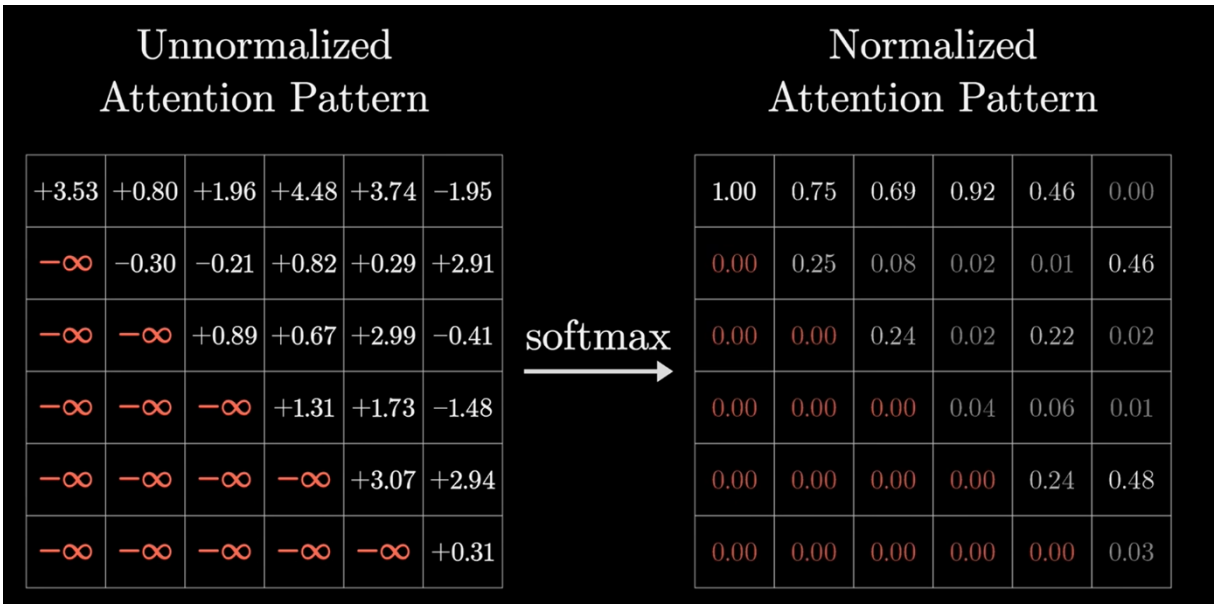
Nasıl Çalışır? (Teknik Süreç)

1. **Skorların Hesaplanması:** Her zamanki gibi QxK^T çarpımı yapılır. Elimizde NxN boyutunda bir matris oluşur.
2. **Maskeleme (En Kritik Adım):** Matrisin üst üçgen kısmı (diyagonalin üstünde kalan, gelecekteki kelimeleri temsil eden hücreler) eksi sonsuz değerleriyle doldurulur.
3. **Softmax:** Softmax fonksiyonu eksi sonsuz değerlerini 0'a dönüştürür.
4. **Sonuç:** Kelimeler arasındaki ağırlıklar hesaplanırken, gelecekteki kelimelerin ağırlığı otomatik olarak 0 olur.

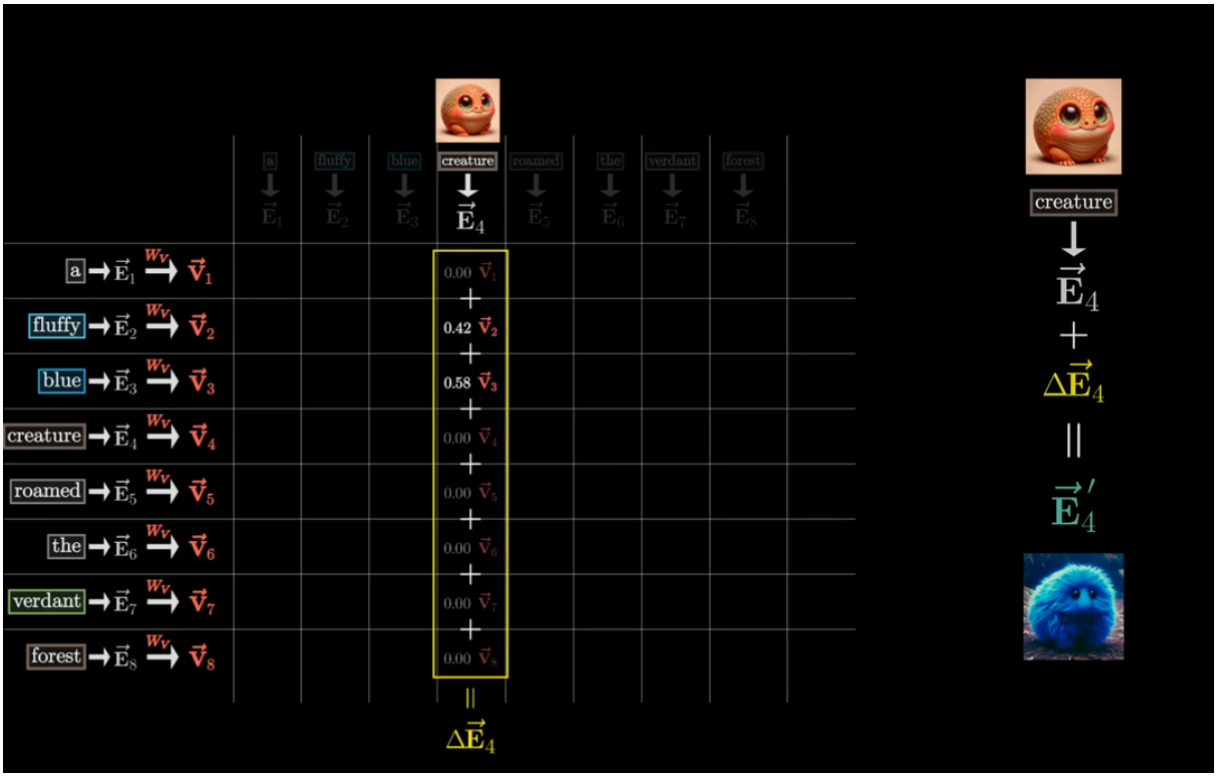
Bir Örnekle Açıklayalım

Cümlemiz: **"Yapay zeka dünyayı..."** olsun.

- **"Yapay"** kelimesi işlenirken: Sadece kendisine bakabilir. ("zeka" ve "dünyayı" maskelenmiştir).
- **"Zeka"** kelimesi işlenirken: "Yapay" ve "zeka" kelimelerine bakabilir. ("dünyayı" maskelenmiştir).
- **"Dünyayı"** kelimesi işlenirken: Kendisinden önceki tüm kelimelere ("Yapay", "zeka", "dünyayı") bakarak bir sonraki kelimeyi (örneğin: "değiştiriyor") tahmin etmeye çalışır.



- Burada next token prediction için masking işlemini görüyoruz.



Dropout

Self-attention mekanizmasında **dropout**, modelin eğitim sırasında belirli "dikkat bağlarına" aşırı güvenmesini (ezberlemesini) engelleyen bir düzenlileştirme (regularization) tekniğidir. Bazı kelimelerin üssünü rastgele kapatmak gibi mi overfitting'i önlemek için.

Özellikle Transformer yapılarında dropout genellikle iki ana noktada uygulanır:

1. Attention Weights (Dikkat Ağırlıkları) Üzerinde

Softmax işleminden hemen sonra, elde edilen attention_weights matrisindeki bazı değerler rastgele olarak sıfıra çekilir.

- **İşleyiş:** Diyelim ki "banka" kelimesi "para" kelimesine %90 ağırlık veriyor. Dropout bu bağı geçici olarak koparır.
- **Amacı:** Modelin sadece tek bir kelimeye odaklanmak yerine, alternatif kelimelerden de bilgi toplamayı öğrenmesini sağlar. Bu, modelin **overfitting** (aşırı öğrenme/ezberleme) yapmasını engeller ve genelleme yeteneğini artırır.

2. Context Vector (Bağlamsal Vektör) Çıkışında

Ağırlıklı toplam yapılıp context_vector elde edildikten sonra, bu vektör bir sonraki katmana (genellikle Residual Connection öncesi) gönderilmeden önce tekrar dropout'a girer.

- **Amacı:** Karmaşık ağ yapısı içindeki nöronların birbirine aşırı bağımlı hale gelmesini (co-adaptation) önlemektir.

Özetle Dropout'un Faydaları Ezberlemeyi Önler, Daha Dayanıklı Temsiller, Kapasite Kullanımı.

Multi-Head Attention

Multi-Head Attention kavramını, tek bir devasa göz yerine, farklı açılardan bakan birçok küçük gözün birleşimi gibi düşünebilirsin. Senin yazdığın o tekli süreci (Single-Head) daha iyi anlamak için bir hikaye üzerinden gidelim. Cümlede öğrenilmesi gerekn çok fazla ilişki türü vardır bunun için kullanabiliriz. Single attention bir bağlam oluştururken multi-head attention birden fazla bağlam oluşturuyor.

Eğitim bittiğinde elinde olan o Wq, Wk, Wv matrisleri aslında tek bir blok halindedir. Ancak model çalışırken bu matrisleri **baş sayısına (head count)** böler.

- **Örnek:** Eğer embedding boyutun **4** ise (senin paylaştığın koddaki gibi) ve **2 baş** kullanıyorsan; 4 boyutlu veri, her biri 2 boyutlu olan iki farklı parçaya ayrılır.
- Bu sayede birinci baş verinin ilk yarısına, ikinci baş ise diğer yarısına odaklanır.

İşte işin "zeka" kısmı burasıdır. Her bir parça (baş), kendi içinde bağımsız bir "dikkat" hesabı yapar. Bu başlar eğitim sırasında otomatik olarak farklı görevler üstlenir:

- **1. Baş (Dilbilgisi):** "Bu kelime bir fiil mi, öznesi nerede?" gibi yapısal bağlamlara bakabilir.
- **2. Baş (Anlam):** "Bu kelime mecaz mı, gerçek anlam mı?" gibi anlamsal bağlamları inceleyebilir.

- **3. Baş (Referans):** "Cümledeki 'o' zamiri kimi işaret ediyor?" sorusuna yanıt arayabilir.

Her baş kendi bağlamına baktıktan sonra bir sonuç üretir. Ancak bu sonuçlar "yarım" bilgilerdir.

1. **Concatenate (Yapıştırma):** Farklı bağlamlardan gelen bu küçük vektörler yan yana dizilir.
2. **Linear Projection (Wo):** Yan yana dizilen bu parçalar, son bir Wo (Output) matrisi ile çarpılarak tek bir anlamlı ve bütünleşik vektöre dönüştürülür.

Evet, Wq, Wk, Wv matrisleri bölünür; her parça **farklı bir uzmana** (head) verilir. Bu uzmanların her biri cümleye "başka bir soru" sorar. En sonunda tüm bu uzmanların görüşleri birleştirilerek kelimenin o cümledeki **nihai ve en zengin anlamı** (Context Vector) oluşturulur.

1. Neden "Multi" (Çoklu)?

Diyelim ki şu cümleyi analiz ediyoruz: **"Ali, topu hızla kaleye fırlattı."**

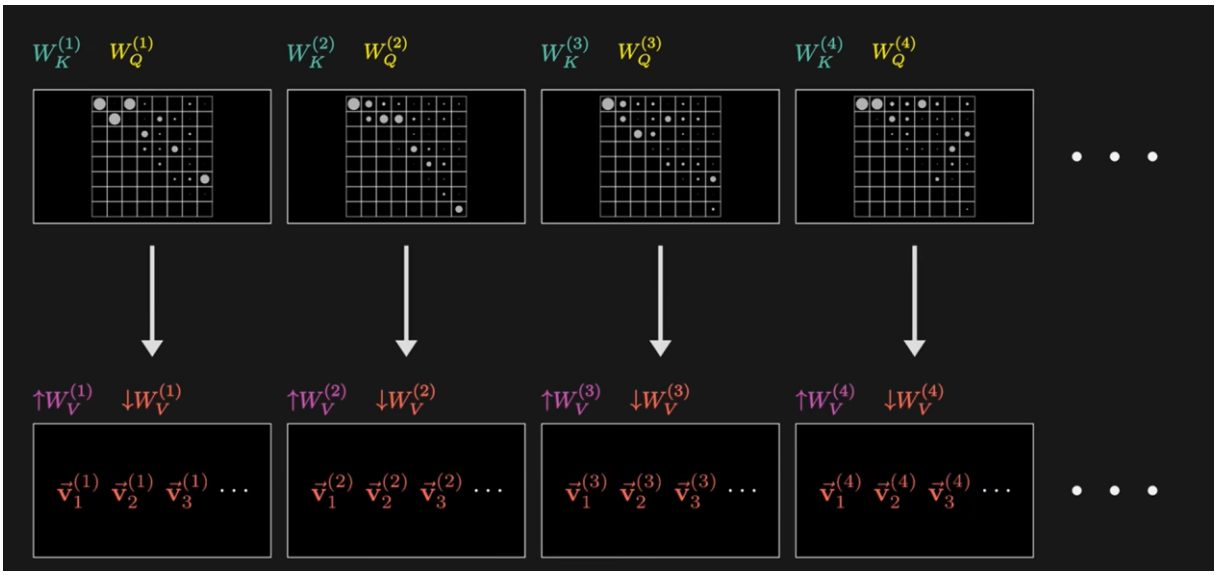
Eğer sadece bir tane attention başın olsaydı, bu baş muhtemelen en güçlü ilişkiye odaklanırdı (örneğin: Ali -> fırlattı). Ancak bir cümlede öğrenilmesi gereken çok fazla farklı ilişki türü vardır:

- **Baş 1 (Özne-Fiil):** "İşi kim yaptı?" sorusuna odaklanır (Ali -> fırlattı).
- **Baş 2 (Nesne-Fiil):** "Neyi fırlattı?" sorusuna odaklanır (Top -> fırlattı).
- **Baş 3 (Zarf-Fiil):** "Nasıl fırlattı?" sorusuna odaklanır (Hızla -> fırlattı).
- **Baş 4 (Yer-Yön):** "Nereye fırlattı?" sorusuna odaklanır (Kale -> fırlattı).

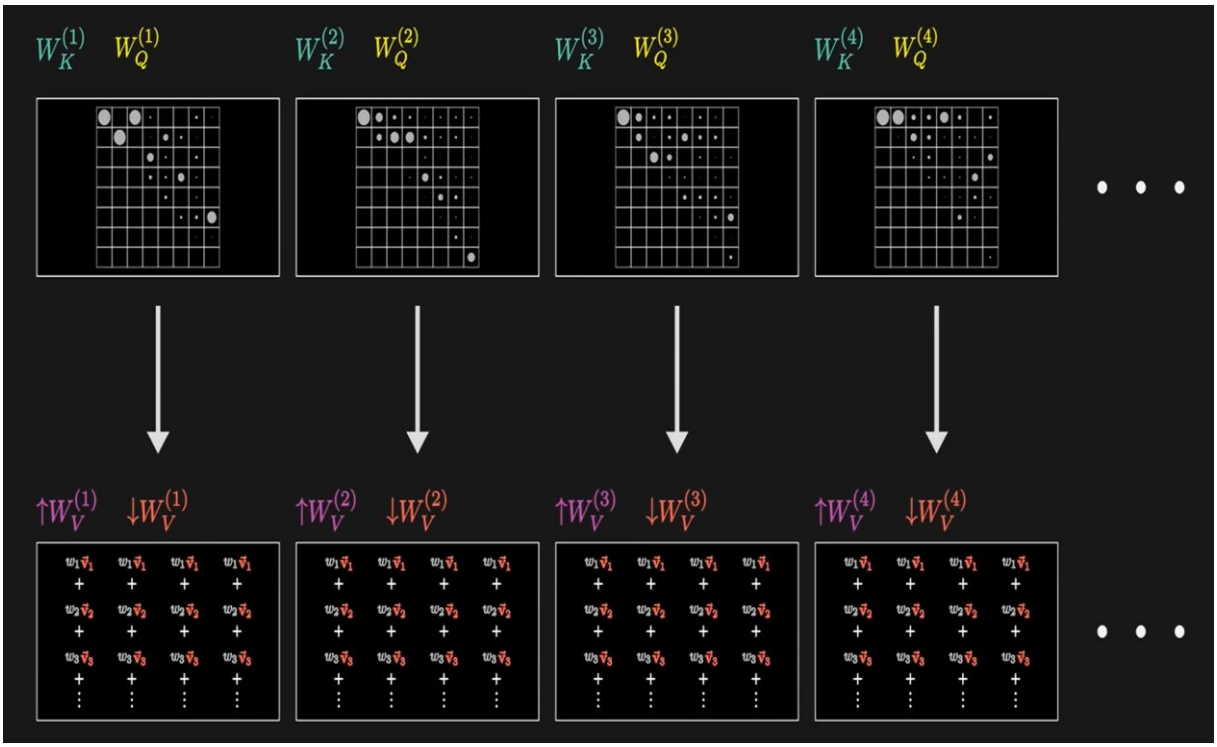
İşte Multi-Head yapısı, bu soruların hepsine **aynı anda** farklı başlarla cevap aramaktır.

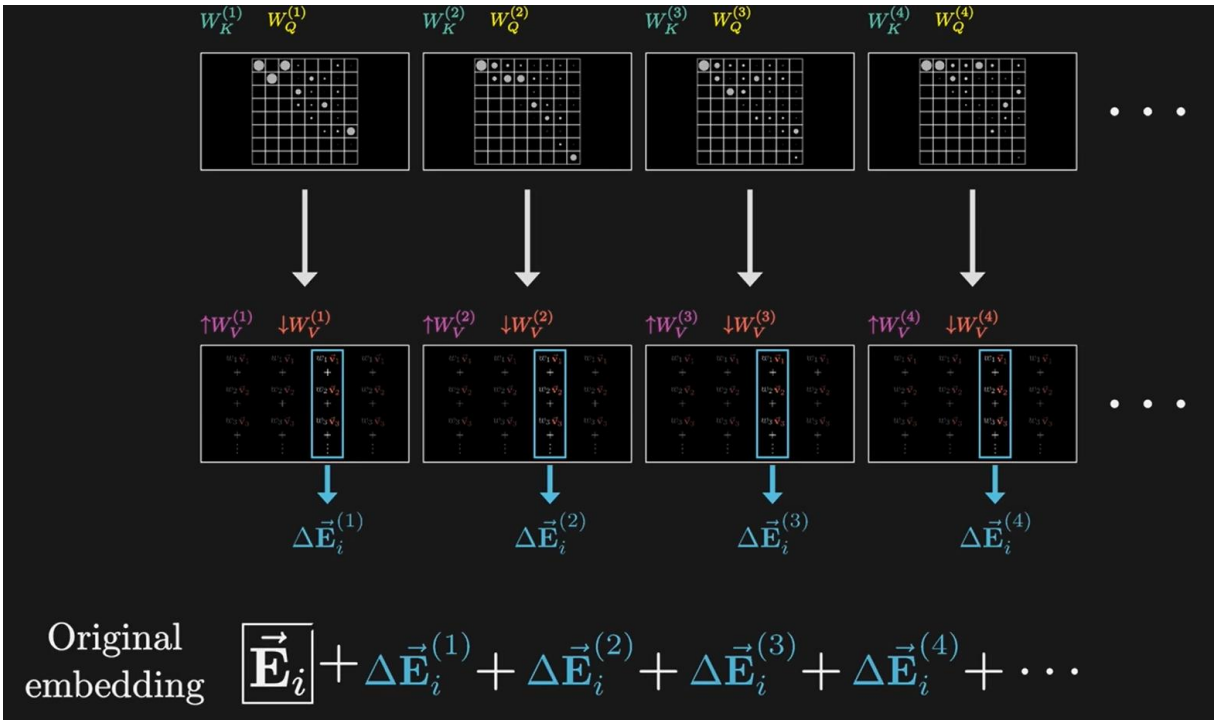
Gerçek Modellerde Multi-Head Attention

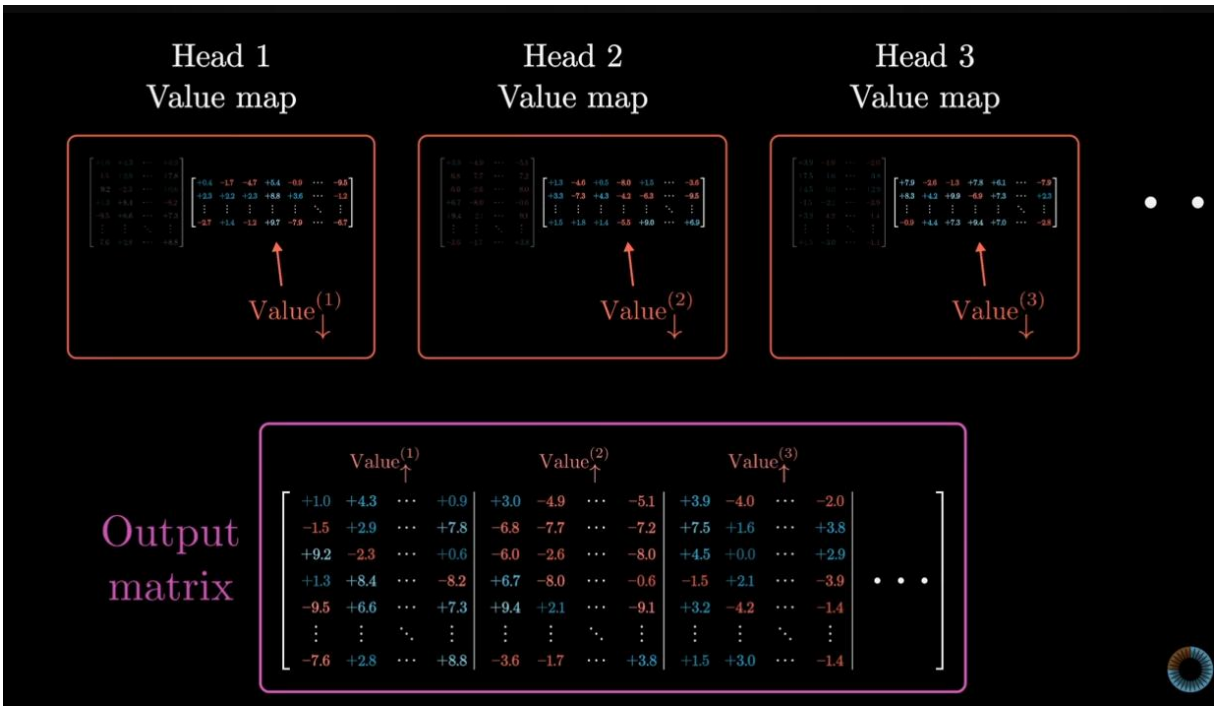
Günümüzdeki LLM'lerin (GPT-4, Llama 3, Mistral vb.) neredeyse tamamında **Causal Attention** ve **Multi-Head Attention (MHA)** beraber kullanılır.



Örneğin nazı GPT modellerin 96 farklı Wq, Wk ve Wv matrisleri vardır.







Model, dizideki kelimeler veya tokenlar arasındaki farklı ilişkileri (örneğin biri dil bilgisi ilişkisine bakarken, diğeri zamansal ilişkiye bakar) yakalayabilmek için veriyi paralel olarak farklı alt uzaylara böler.

- **Value map:** Her bir kafanın kendi sorgu (Query) ve anahtar (Key) matrislerini çarpıp softmax'ten geçirdikten sonra, kendi değer V matrisiyle çarpılması sonucu elde ettiği benzersiz **Value** haritasıdır.

- Görselde üç farklı kafa (Value{(1)}, Value{(2)}, Value{(3)}) ve devamında diğer kafaların olduğunu belirten üç nokta ($\dots$) görüyoruz.
- Dikkat ederseniz her bir matrisin içindeki sayılar (renklerle de belirtilmiş) tamamen birbirinden farklıdır; çünkü her kafa verinin farklı bir yönüne odaklanmıştır.

MultiHead Attention Örneği:

Senaryomuz ve Gerçekçi Boyutlarımız

- **Cümlemiz:** "Yapay zeka" (2 kelime) $\rightarrow$ Kelime Sayısı (N) = **2**
- **Modelin Orijinal Boyutu (d_{model}): 768** (Sektör standardı)
- **Kafa Sayısı (Head - h): 96** (İstediğin gibi, devasa bir model!)
- **Kafa Başına Düşen Boyut (d_k ve d_v):** $768/96 = 8$ *(Yani her kafa kelimeleri 8 boyutlu alt uzaylarda inceleyecek).*

1. Adım: Girdi Matrisi (X) - [Doğruladığın Adım]

"Yapay" ve "zeka" kelimelerinin 768 boyutlu vektörleri ile pozisyon (sıra) bilgileri matris seviyesinde toplanır. Elimizde 2×768 boyutunda devasa bir X matrisi oluşur:

$$X = \begin{bmatrix} [\text{Yapay}] \rightarrow 0.12, & -0.45, & 0.89, & \dots & (768 \text{ tane sayı}) \\ [\text{zeka}] \rightarrow -0.23, & 0.67, & -0.11, & \dots & (768 \text{ tane sayı}) \end{bmatrix}$$

2. Adım: 96 Farklı Kafaya Dağılım ve Projeksiyon

Elimizde tam **96 takım** W_Q, W_K, W_V ağırlık matrisi var. Her bir kafa matrisinin boyutu 768×8'dir. X matrisini bu ağırlıklarla çarparak her kafa için **8 boyutlu** sorgu, anahtar ve değer matrislerini üretiriz.

Gözümüzde canlanması için **Kafa 1** ve **Kafa 96**'nın ürettiği gerçekçi sayılara bakalım:

Head 1 (Örneğin: Kelimelerin dil bilgisi ilişkisine bakan kafa)

$$Q_1 = X \cdot W_{Q1} = \begin{bmatrix} 0.5, & -0.2, & 0.1, & \dots & (8 \text{ sayı}) \\ -0.1, & 0.8, & -0.3, & \dots & (8 \text{ sayı}) \end{bmatrix} \quad (2 \times 8)$$

$$K_1 = X \cdot W_{K1} = \begin{bmatrix} 0.4, & 0.1, & -0.5, & \dots & (8 \text{ sayı}) \\ 0.2, & -0.6, & 0.9, & \dots & (8 \text{ sayı}) \end{bmatrix} \quad (2 \times 8)$$

$$V_1 = X \cdot W_{V1} = \begin{bmatrix} 1.2, & 0.5, & -0.1, & \dots & (8 \text{ sayı}) \\ -0.4, & 1.1, & 0.6, & \dots & (8 \text{ sayı}) \end{bmatrix} \quad (2 \times 8)$$

Head 96 (Örneğin: Kelimelerin eş anlam/bağlam ilişkisine bakan kafa)

$$Q_{96} = X \cdot W_{Q96} = \begin{bmatrix} -0.7, & 0.3, & \dots & (8 \text{ sayı}) \\ 0.5, & -0.1, & \dots & (8 \text{ sayı}) \end{bmatrix}, \quad K_{96} = \dots, \quad V_{96} =$$

3. Adım: Nokta Çarpım ve $\sqrt{d_k}$ Ölçeklendirme

Şimdi **Kafa 1** kendi içinde kelimelerin birbiriyle ilişkisini bulmak için Q_1 ve K_1^T (8×2) matrislerini çarpar. Çıkan sonucu hemen kafa boyutu olan $d_k = 8$'in kareköküne ($\sqrt{8} \approx 2.82$) böler.

Diyelim ki bu çarpım ve bölme sonucunda **Kafa 1** için şu ham skor matrisi çıktı:

$$\text{Ölçeklenmiş Skorlar (Head 1)} = \begin{bmatrix} \text{Yapay-Yapay: } 1.1 & \text{Yapay-zeka: } 4.5 \\ \text{zeka-Yapay: } 3.8 & \text{zeka-zeka: } 0.8 \end{bmatrix}$$

4. Adım: Softmax ile Olasılık Haritası

Bu skorları Softmax fonksiyonuna sokarak kelimelerin birbirine olan dikkat yüzdelerini hesaplıyoruz:

$$\text{Attention Weights (Head 1)} = \text{Softmax} \begin{bmatrix} 1.1 & 4.5 \\ 3.8 & 0.8 \end{bmatrix} = \begin{bmatrix} \%4 & \%96 \\ \%95 & \%5 \end{bmatrix} = \begin{bmatrix} (\\ (\end{bmatrix}$$

- **Yorum:** Kafa 1 diyor ki: "Yapay" kelimesinin anlamını çözmek için %96 oranında "zeka" kelimesine bakmalıyım!

5. Adım: Değer (V) Matrisi ile Çarpım (Anlam Süzgeci)

Şimdi bu %96 ve %95'lik devasa dikkat oranlarını, 2. adımda ürettiğimiz gerçek anlamları barındıran 8 boyutlu V_1 matrisi ile çarpıyoruz.

$$\text{Head 1 Çıktısı} = \begin{bmatrix} 0.04 & 0.96 \\ 0.95 & 0.05 \end{bmatrix} \cdot \begin{bmatrix} 1.2, & 0.5, & \dots & (8 \text{ sayı}) \\ -0.4, & 1.1, & \dots & (8 \text{ sayı}) \end{bmatrix}$$

Bu çarpım bittiğinde **Kafa 1**, "Yapay" kelimesinin içine "zeka"nın anlamını yoğun bir şekilde akıtmış olur ve elimizde 2×8 boyutunda rafine bir matris kalır:

$$\text{Head 1 Çıktısı} = \begin{bmatrix} [\text{Yapay'ın yeni hali}] \rightarrow -0.33, & 1.07, & \dots & (8 \text{ sayı}) \\ [\text{zeka'nın yeni hali}] \rightarrow 1.12, & 0.53, & \dots & (8 \text{ sayı}) \end{bmatrix}$$

6. Adım: 96 Kafanın Birleştirilmesi (Concatenation) - [İlk Görseldeki Mor Kutu]

İşte zurnanın zırt dediği, o muazzam birleşme anı! 96 kafanın tamamı (Head 1'den Head 96'ya kadar) kendi süzme işlemlerini bitirdi. Hepsinin elinde 2×8 boyutunda birer matris var.

Bu 96 matrisi yan yana yapıştırıyoruz:

$$\text{Birleştirilmiş Matris} = [\text{Head 1} \quad | \quad \text{Head 2} \quad | \quad \dots \quad | \quad \text{Head 96}]$$

Boyut hesabını yapalım:

- Satır sayısı (Kelime Sayısı) değişmedi: **2**
- Sütun sayısı: 96 kafa $\times$ kafa başına 8 sayı = **768**
- **Yeni Matris Boyutu:** 2×768

Gördüğün gibi model, parçalara ayırdığı o 768 boyutlu büyük dünyayı, 96 farklı açıdan işlenmiş ve olgunlaşmış olarak **tekrar orijinal boyutuna (2×768) geri getirdi.**

Final: W_O Doğrusal Projeksiyon Matrisi

Son olarak, yan yana yapıştırılan bu 96 kafanın birbirleriyle tamamen kaynaşması ve tek bir potada erimesi için model, bu matrisi 768×768 boyutundaki devasa bir W_O (Output) ağırlık matrisiyle çarpar:

$$\text{Nihai MHA Çıktısı} = \text{Birleştirilmiş Matris } (2 \times 768) \cdot W_O(768 \times 768) =$$

1. W_O Matrisi Nedir ve Tam Olarak Ne İş Yapar?

Bir önceki adımda 96 kafadan gelen 2×8 boyutundaki matrisleri yan yana yapıştırarak 2×768 boyutunda büyük bir matris elde etmiştik.

Ancak burada büyük bir problem vardır: Matris yan yana yapışmış olsa da, **1. kafanın ürettiği bilgiler ile 96. kafanın ürettiği bilgiler henüz birbiriyle konuşmamıştır**. Sadece yan yana duruyorlardır.

- **Bilgi Karıştırma (Linear Projection):** W_O matrisi (768×768), bu yan yana duran 96 farklı bakış açısını matris çarpımı yoluyla **birbiriyle harmanlar**. 1. kafanın bulduğu dil bilgisi kuralı ile 96. kafanın bulduğu anlam ilişkisi bu çarpım sayesinde birbiriyle etkileşime girer.
- **Boyut Uyumluluğu:** Modelin sonraki katmanları (örneğin Transformer içindeki o meşhur "Residual Connection / Artık Bağlantılar") girdi olarak orijinal boyutta veri bekler. W_O matrisi, çıktının biçimini ve yapısını modelin geri kalanıyla kusursuz bir şekilde eşitleyen son dokunuştur.

2. Bu Matrisi Kim Belirler? (Yapay Zeka Mimarisi)

Bu matrisin **boyutlarını ve varlığını** yapay zeka modelini tasarlayan **mühendisler (mimari)** belirler.

- Mühendis modeli kodlarken der ki: *"Ben bu modelde gizli boyutu (d_{model}) 768 seçtim. Dolayısıyla Multi-Head Attention'dan çıkarken veriyi tekrar karıştıracak olan W_O matrisinin boyutu da otomatik olarak 768×768 olmak zorundadır."*

Ancak mühendis, bu matrisin **içindeki sayıların (ağırlıkların) ne olacağını asla elle yazmaz.**

Boyut Kıyaslaması:

- **Kelime Sayısı (N):** 2 ("Yapay", "zeka")
- **Model Boyutu (d_{model}):** 768
- **Kafa Sayısı (h):** 96
- **Kafa Başına Boyut (d_k veya d_v):** $768/96 = 8$

1. Giriş Kapısı: Girdi Matrisi (X)

Kelimelerin embedding ve pozisyon vektörleri birleşti. Elimizde yan yana 768 sütunu, alt alta 2 satırı olan bir dikdörtgen var.

- **Boyut:** 2×768 *(2 Kelime × 768 Özellik)*

2. Parçalama Odası: Q, K, V Üretimi

Bu 2×768'lik matrisi, her kafanın kendi W_Q, W_K, W_V ağırlık matrisleriyle çarpıyoruz. Bu ağırlık matrislerinin görevi 768'i alıp 8'e indirmektir. Bu yüzden boyutları 768×8'dir.

Çarpım kuralını hatırlayalım: $(A \times B) \times (B \times C) = A \times C$

$$\mathbf{X} \cdot \mathbf{W_Q} \rightarrow (2 \times 768) \times (768 \times 8) = \mathbf{2 \times 8}$$

Her bir kafa (96 kafanın hepsi) kendi içinde 2×8 boyutunda 3 adet matris (Q, K, V) üretir.

- **Şu anki Durum:** Elimde 96 adet 2×8 boyutunda Q matrisi var.

3. Hesaplaşma Masası: $Q \cdot K^T$ ve Softmax

Kelimelerin birbiriyle ilişkisini bulmak için her kafa kendi Q (2×8) ve K'sının transpozunu (K^T yani 8×2) çarpar.

$$(2 \times 8) \times (8 \times 2) = \mathbf{2 \times 2}$$

- **Boyut:** 2×2 *(Kelime Sayısı × Kelime Sayısı)*
- **Yorum:** Bu matris artık 8 boyutu kaybetti. Sadece 1. kelimenin 1. ve 2. kelimeyle olan ilişkisini gösteren saf bir haritadır. Softmax'ten geçince de boyutu 2×2 olarak kalır.

4. Anlam Yükleme: Softmax · V

Elimizdeki 2×2'lik ilişki haritasını, 2. adımda ürettiğimiz 2×8 boyutundaki gerçek anlam (V) matrisiyle çarpıyoruz.

$$(2 \times 2) \times (2 \times 8) = \mathbf{2 \times 8}$$

- **Boyut:** 2×8
- **Sonuç:** Her kafa, kendi 8 boyutlu dünyasına geri döndü. Ama artık bu 8 sayının içinde cümlenin diğer kelimelerinin de etkisi var.

5. Birleştirme Bandı (Concatenation)

96 kafanın her biri işini bitirdi ve elinde 2×8 boyutunda bir çıktı var. İkinci sorundaki mor kutuyu hatırla: Bu 96 tane matrisi **yan yana yapıştırıyoruz.**

- Satır sayısı değişmedi, hala 2 kelimeyle çalışıyoruz $\rightarrow$ **2**
- Sütun sayısı: Yan yana 96 tane 8 sütunlu matris dizildi $\rightarrow 96 \times 8 = \mathbf{768}$
- **Birleştirilmiş Matris Boyutu:** 2×768

Bak, en baştaki X matrisinin boyutuna (2×768) geri döndük!

6. Çıkış Kapısı: W_O ile Çarpım

Son sorunda sorduğun o meşhur W_O matrisi devreye giriyor. Boyutu 768×768. Yan yana yapıştırdığımız o dev matrisi bu W_O ile çarpıyoruz:

$$\text{Birleştirilmiş Matris} \cdot W_O \rightarrow (2 \times 768) \times (768 \times 768) = \mathbf{2 \times 768}$$

Kuş Bakışı Özet Tablosu

İşlem Adımı	Yapılan İşlem	Matris Boyut Değişimi
1. Giriş	Ham Cümle Matrisi (X)	2×768
2. İzdüşüm	X ile W_Q, W_K, W_V çarpımı	2×8 (Kafa başına)
3. Dikkat Skorları	$Q \cdot K^T$ çarpımı	2×2
4. Bağlam Filtresi	$\text{Softmax} \cdot V$ çarpımı	2×8 (Kafa başına)
5. Birleştirme	96 kafayı yan yana dizme	2×768
6. Final Projeksiyon	Çıktıyı W_O (768×768) ile çarpma	2×768

Residual Connection ve Layer Normalization

Multi-Head Attention (Çok Kafalı Dikkat) bloğundan başarıyla çıktın! Elinde artık 96 kafanın süzgecinden geçmiş ve Wo matrisiyle tamamen birbiriyle harmanlanmış **2x768** boyutunda harika bir bağlamsal matris var.

Şimdi Transformer mimarisinin bir sonraki büyük odasına, yani **"Residual Connection (Artık Bağlantı) ve Katman Normalizasyonu (Layer Normalization)"** aşamasına

geçiyoruz. Bu adımda matris boyutumuz **hiç değişmeyecek (2x768 kalacak)** ancak modelin sağlıklı eğitilebilmesi için çok kritik iki operasyon yapılacak:

1. Adım: Residual Connection (Kestirme Yol / Ekleme)

Model der ki: *"Evet, Multi-Head Attention'da kelimeleri birbiriyle harika şekilde eşleştirdim ama bunu yaparken kelimelerin en baştaki ham, saf anlamlarını biraz bulandırmış olabilirim."*

Bu yüzden, en başta bu bloğa giren ham **X matrisi (2x768)** alınır ve Multi-Head Attention'dan taze çıkan **Çıktı matrisine (2x768)** doğrudan matematiksel olarak **eklenir (toplanır)**.

Yeni Matris= X + {MHA Çıktısı}

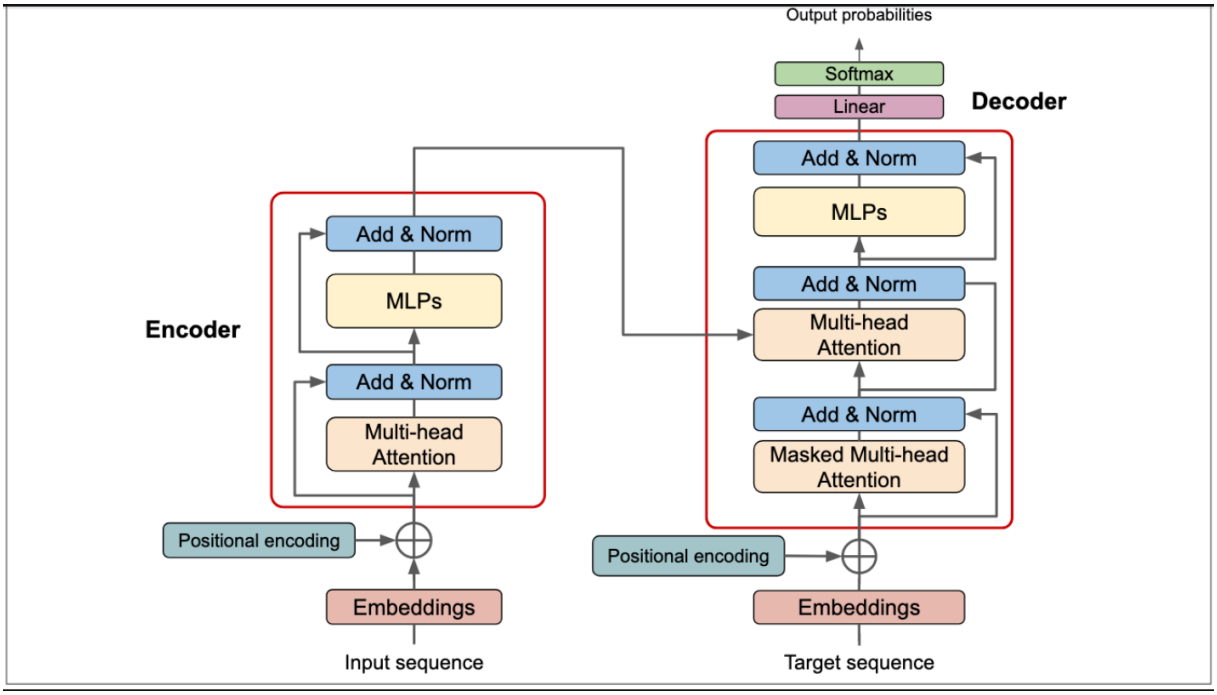
Amacı: Bu kestirme yol (skip connection), gradyanların ağın derinliklerinde kaybolmadan (vanishing gradient) çok daha rahat akmasını sağlar ve ham bilginin unutulmasını engeller.

2. Adım: Layer Normalization (Katman Normalizasyonu)

Toplama işleminden sonra sayılar yine biraz büyümüş veya dengesizleşmiş olabilir. Sayıları ehlileştirmek için matris **LayerNorm** işlemine sokulur.

- Her bir kelimenin 768 boyutlu vektörü kendi içinde incelenir. Bu 768 sayının ortalaması ve varyansı hesaplanarak sayılar **0 ortalama ve 1 varyans** olacak şekilde yeniden ölçeklendirilir.

Transformer Mimarisi (Encoder - Decoder)



1. Giriş Katmanı (Verinin Hazırlanması)

Şemanın en altında, hem Encoder hem de Decoder girişinde aynı mekanizmayı görüyoruz: **Embeddings** ve **Positional Encoding**.

- **Embeddings (Girdi Vektörleri):** Metin tokenlara (kelime parçalarına) bölündükten sonra, her token'ın anlamsal özelliklerini içeren o meşhur büyük vektörlere (d_model = 768) dönüştürüldüğü yerdir.
- **Positional Encoding (Pozisyon Bilgisi):** Transformer modeli kelimeleri arka arkaya değil, aynı anda (paralel) işler. Kelimelerin cümledeki sırasını (hangisi önce, hangisi sonra) modele anlatabilmek için kelime vektörlerine sinüs/cosinus dalgalarından üretilen benzersiz pozisyon sinyalleri eklenir.
- Şemadaki **artı sembolü**, bu iki matrisin (Anlam + Pozisyon) toplandığını ve içeriye girecek olan nihai X matrisini oluşturduğunu gösterir.

2. Sol Taraf: Encoder (Kodlayıcı) Katmanı

Encoder'ın görevi, girdi dizisindeki ("Yapay zeka") kelimelerin birbirleriyle olan ilişkilerini **çift yönlü (her kelime sağındakini de solundakini de görecek şekilde)** incelemek ve bağlamsal olarak en zengin anlam haritasını çıkarmaktır.

1. **Multi-head Attention:** Gönderdiğin ikinci fotoğraftaki mor kutuda olduğu gibi, veri paralel olarak 96 farklı kafaya bölünür. Kelimeler birbirine sorular sorar (Q, K, V). Burada maskeleme yoktur, "Yapay" kelimesi "zeka"yı, "zeka" kelimesi de "Yapay"ı tamamen görebilir.
2. **Add & Norm (Residual + LayerNorm):** * **Add:** Multi-head attention bloğunun yanından dolaşıp yukarı çıkan oku görüyorsun. Bu ok, katmanın girişindeki ham veriyi alıp çıktıyla toplar (**Residual Connection**).
    - **Norm:** Toplanan bu veriler sayısal olarak dengelenmesi için **Layer Normalization** işlemine sokulur.
3. **MLPs / Feed-Forward (Yapay Sinir Ağı):** Normalizasyondan çıkan veri, kelimelerin kendi içlerinde derin matematiksel ilişkiler kurabilmesi için önce 4 kat genişleyen (3072 boyutuna), ardından tekrar eski boyutuna (768) dönen iki katmanlı lineer ağdan geçer.
4. **İkinci Add & Norm:** MLP çıkışında veri tekrar bir kestirme yolla (residual) toplanır ve normalize edilir.

Kritik Detay: Encoder bloğunun en üstünden çıkan ok, sağ taraftaki Decoder bloğunun tam ortasına doğru uzanır. Encoder, "Ben girdi metnini mükemmel şekilde analiz ettim, anlam haritası burada" diyerek bu bilgiyi Decoder'a paslar.

Decoder Katmanı

1. Basamak: Giriş ve Masked Multi-Head Attention

Decoder bloğuna o ana kadar ürettiğimiz hedef kelimeler girer: ["Yapay"].

- Bu kelimeler embedding ve pozisyon eklemesinden geçip matrisleştikten sonra ilk alt bloğa yani **Masked Multi-Head Attention**'a uğrar.
- **Burada ne oluyor?** Bir önceki adımda konuştuğumuz gibi, model sağındaki (gelecekteki) kelimeleri görmesin diye **Causal Masking (-sonsuz)** uygulanır. Kelimeler sadece kendilerinden önce gelen geçmiş kelimelere bakarak bir iç ilişki haritası çıkarır.
- Bu bloktan çıktıktan sonra hemen **Add & Norm** (Residual toplama ve Layer Normalization) uygulanır.
- **Elde kalan:** Geçmiş kelimelerin kendi içindeki ilişkisini çözen, kafa başına 8 boyutlu (toplamda 768) bir matris.

2. Basamak: Ortadaki Esrarengiz Blok (Multi-Head Attention / Cross-Attention)

İşte senin bahsettiğin, **Encoder'dan gelen o okun** tam kalbine daldığı yer burasıdır! Biz buna yapay zekada **Cross-Attention (Çapraz Dikkat)** diyoruz.

Bu bloğun içine **3 tane ok** girer. Dikkatli bakarsan okların ikisi sol taraftan (Encoder'dan), bir tanesi ise aşağıdan (Decoder'ın kendi Masked Attention katmanından) gelir.

Bu 3 ok içeri girdiğinde Q, K, V rolleri şu şekilde paylaştırılır:

- **Sorgu (Q) [Aşağıdan, Decoder'dan gelir]:** Decoder der ki: *"Ben az önce geçmiş kelimeleri işledim ve elimde ['Yapay'] kelimesi var. Şimdi bu kelimenin yanına ne getireceğimi arıyorum (Sorguluyorum)."*
- **Anahtar (K) [Soldan, Encoder'dan gelir]:** Encoder der ki: *"Sen bir şeyler arıyorsun ama bizim orijinal cümlemiz "Artificial intelligence". Bu cümledeki kelimelerin adresleri ve özellikleri bende (Key matrisindeyim)."*
- **Değer (V) [Soldan, Encoder'dan gelir]:** Encoder devam eder: *"Orijinal cümledeki kelimelerin asıl zengin anlam içerikleri de bende saklı (Value matrisindeyim)."*

3. Basamak: Add & Norm ve Feed-Forward (MLP)

Cross-Attention'dan çıkan matris, decoder'ın geri kalanında encoder'dakiyle aynı yoldan geçer:

- **Add & Norm:** Cross-Attention bloğuna giren veri, bloğun çıktısıyla toplanır (residual) ve Layer Normalization uygulanır.
- **Feed-Forward (MLP):** Her token kendi içinde önce 4 kat genişler (3072 boyutuna), sonra tekrar 768'e döner.

- **İkinci Add & Norm:** MLP çıkışı yine kestirme yolla toplanır ve normalize edilir.

Bu üç alt blok (Masked Multi-Head Attention → Cross-Attention → MLP, her birinin ardından Add & Norm) bir **decoder katmanıdır**. Model bu katmandan N tane üst üste dizer; encoder tarafı da aynı şekilde N katmandır.

4. Basamak: Linear + Softmax (Çıkış)

Decoder'ın en üstünden çıkan vektörün artık bir kelimeye dönüşmesi gerekir. Şemanın en üstündeki iki kutu bunu yapar:

- **Linear:** 768 boyutlu vektör, sözlükteki (vocab) her token için bir skor üretecek şekilde sözlük boyutuna genişletilir.
- **Softmax:** Bu skorlar toplamı 1 olan olasılıklara çevrilir (*Output probabilities*).
- **Seçim:** En olası token seçilir (örneğimizde "zeka") ve üretilen kelimelerin sonuna eklenir: ["Yapay", "zeka"].
- **Döngü:** Yeni dizi tekrar decoder'a girer; bitiş token'ı üretilene kadar her turda bir token eklenir.

Örnek:

"Artificial intelligence" cümlesinin "Yapay zeka"ya çevrilmesi:

- **Encoder:** "Artificial intelligence" bir kez okunur ve anlam haritası çıkarılır. Cross-Attention'daki K ve V buradan gelir.
- **1. tur:** Decoder'a başlangıç token'ı girer → Linear + Softmax → "Yapay".
- **2. tur:** Decoder'a ["Yapay"] girer → Masked Attention geçmişe, Cross-Attention encoder'a bakar → "zeka".
- **3. tur:** Decoder'a ["Yapay", "zeka"] girer → bitiş token'ı üretilir, çeviri tamamlanır.

GPT Gibi Modeller: Sadece Decoder

Yukarıdaki encoder–decoder yapısı çeviri gibi görevler için tasarlanmıştır. GPT gibi metin üreten modeller ise yalnızca decoder kısmını kullanır (**decoder-only**):

- **Encoder yok:** Ayrı bir girdi cümlesi olmadığı için Cross-Attention bloğu da yoktur.
- **Her katman:** Causal (Masked) Multi-Head Attention → Add & Norm → Feed-Forward (MLP) → Add & Norm.
- **Çıkış:** Bu katman N kez üst üste dizilir; en üstte Linear + Softmax ile sonraki token tahmin edilir.
- **Üretim:** Seçilen token metnin sonuna eklenir ve döngü tekrarlar. Bu, Next Token Prediction'ın ta kendisidir.